# Embedded Control for Robotics

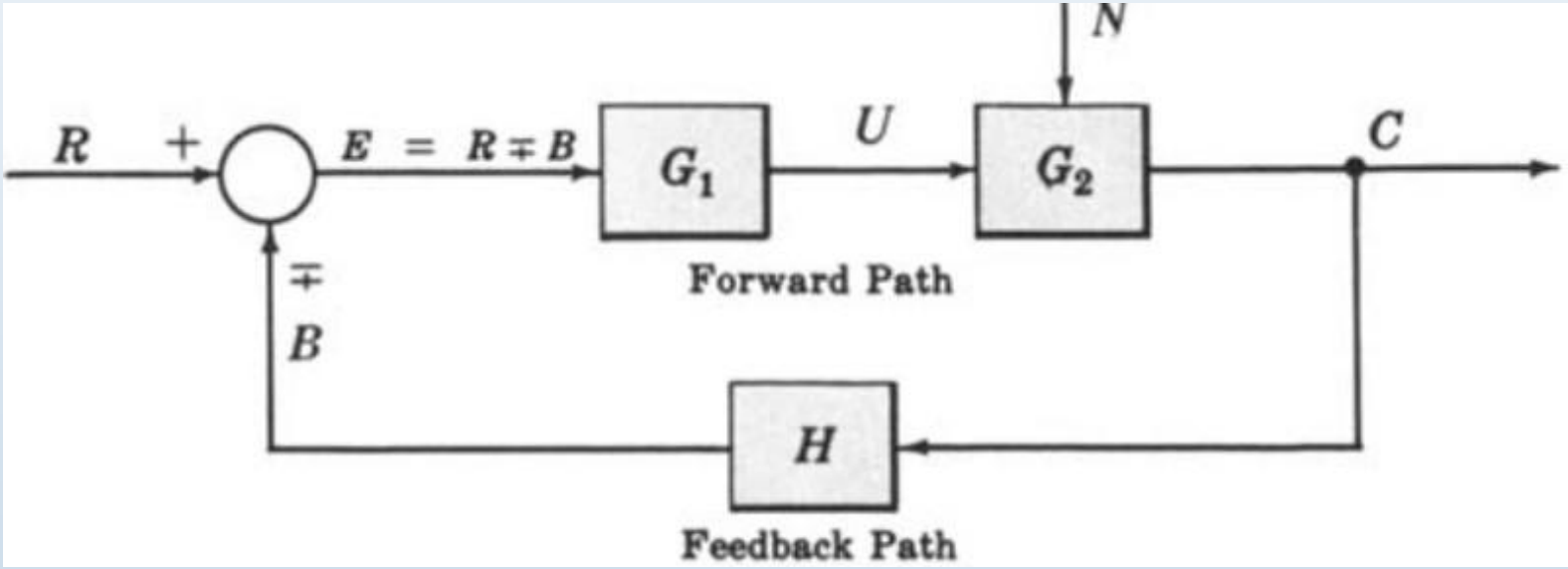
Lecture # 6: **Feedback control system modeling**

Book: Ch 6, 7, FEEDBACK and CONTROL SYSTEMS, Schaum's Outline

Dr. Suleman Khan
CS, NUCES-FAST, Islamabad

# Feedback system





# Canonical Form



- A canonical form refers to a standard or simplified representation of a system's mathematical model, particularly in state-space representation.

- The purpose of canonical forms is to simplify analysis and design by presenting the system equations in a structured and predictable manner.

# Canonical Form



$G \equiv$ direct transfer function $\equiv$ forward transfer function

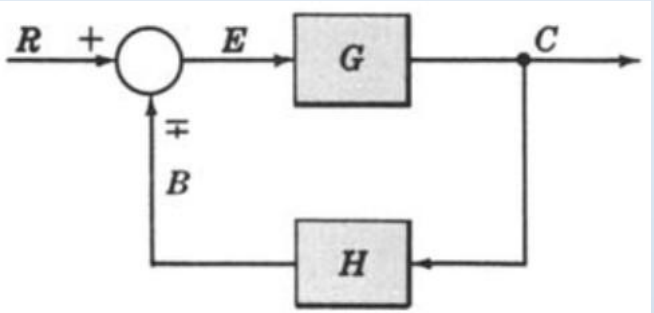
$H \equiv$ feedback transfer function

$GH \equiv$ loop transfer function $\equiv$ open-loop transfer function

$C/R \equiv$ closed-loop transfer function $\equiv$ control ratio

$E/R \equiv$ actuating signal ratio $\equiv$ error ratio

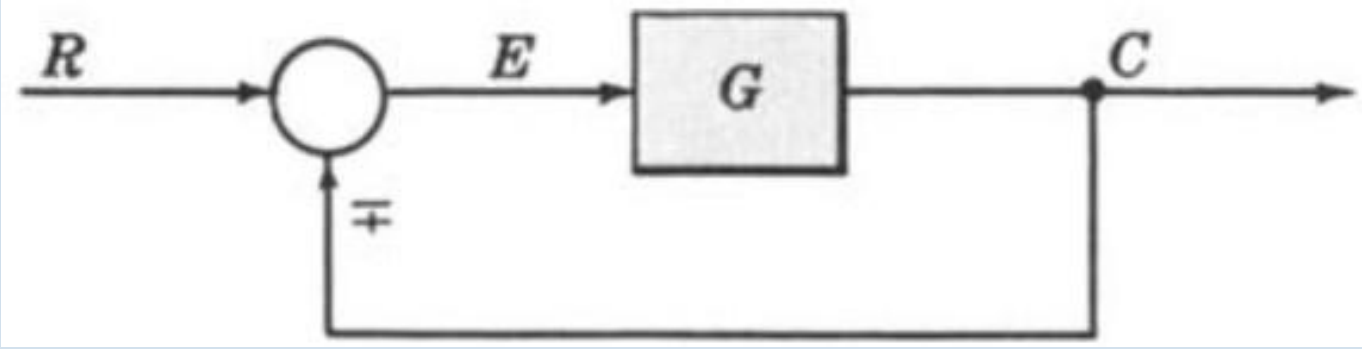
$B/R \equiv$ primary feedback ratio



# Unity Feedback



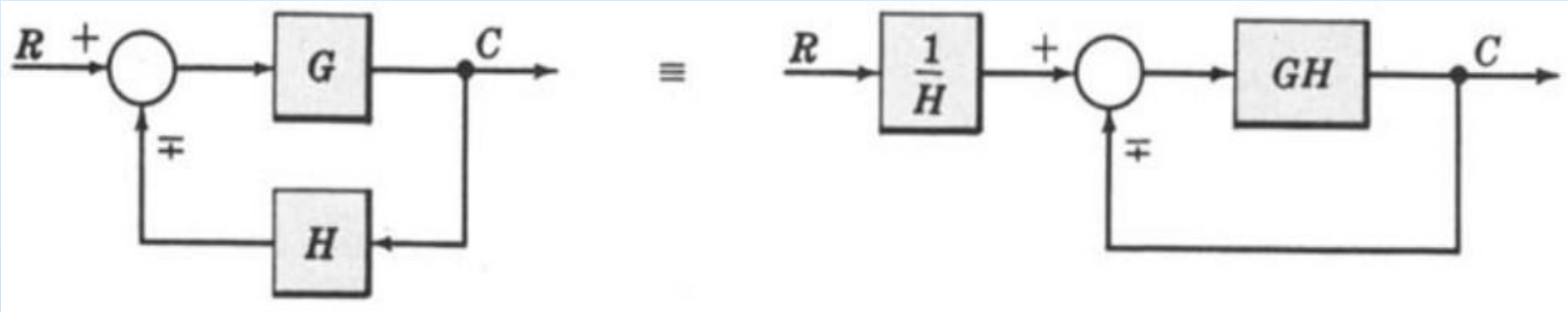
- To make the units same of input and output the feedback system is transformed into a **UNITY FEEDBACK** system
- **The system can be transformed to UNITY FEEDBACK is the transfer function of H(s) is a pure gain**
- **UNITY FEEDBACK:** In which the feedback path **H(s)** =1



# Conversion to Unity Feedback Form

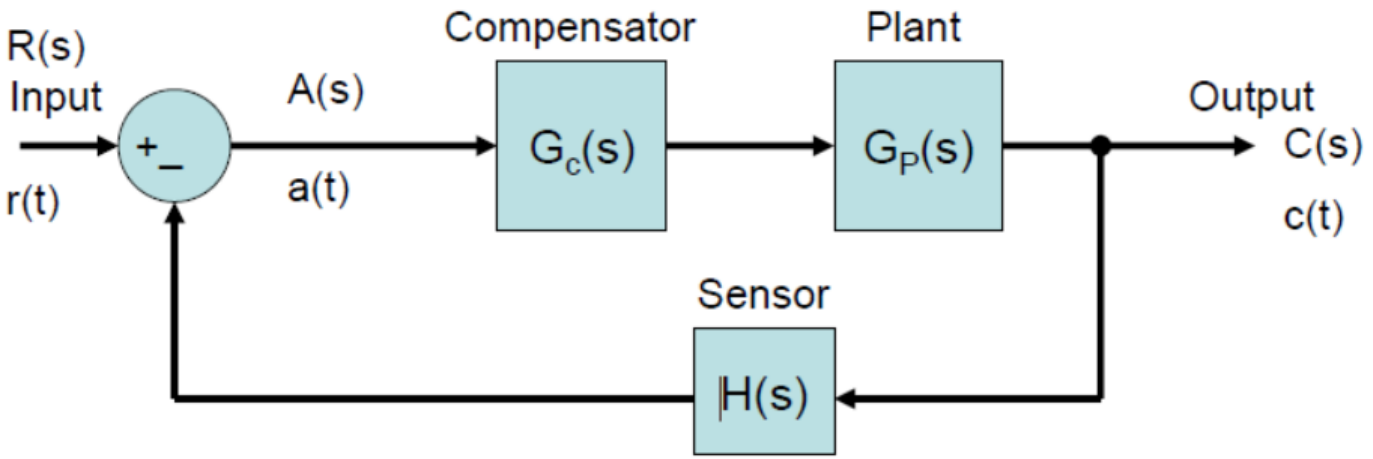


- Canonical form can be converted to Unity Feedback form

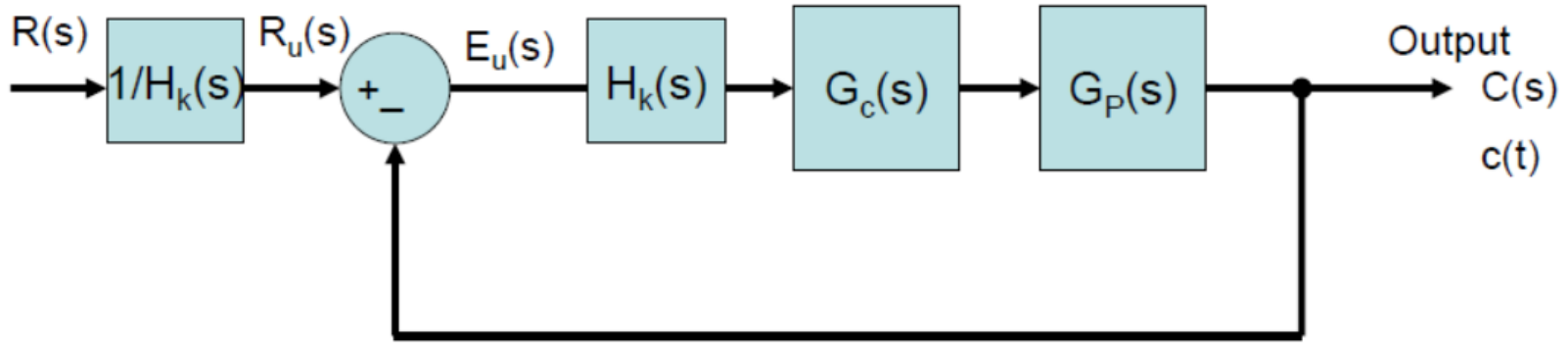


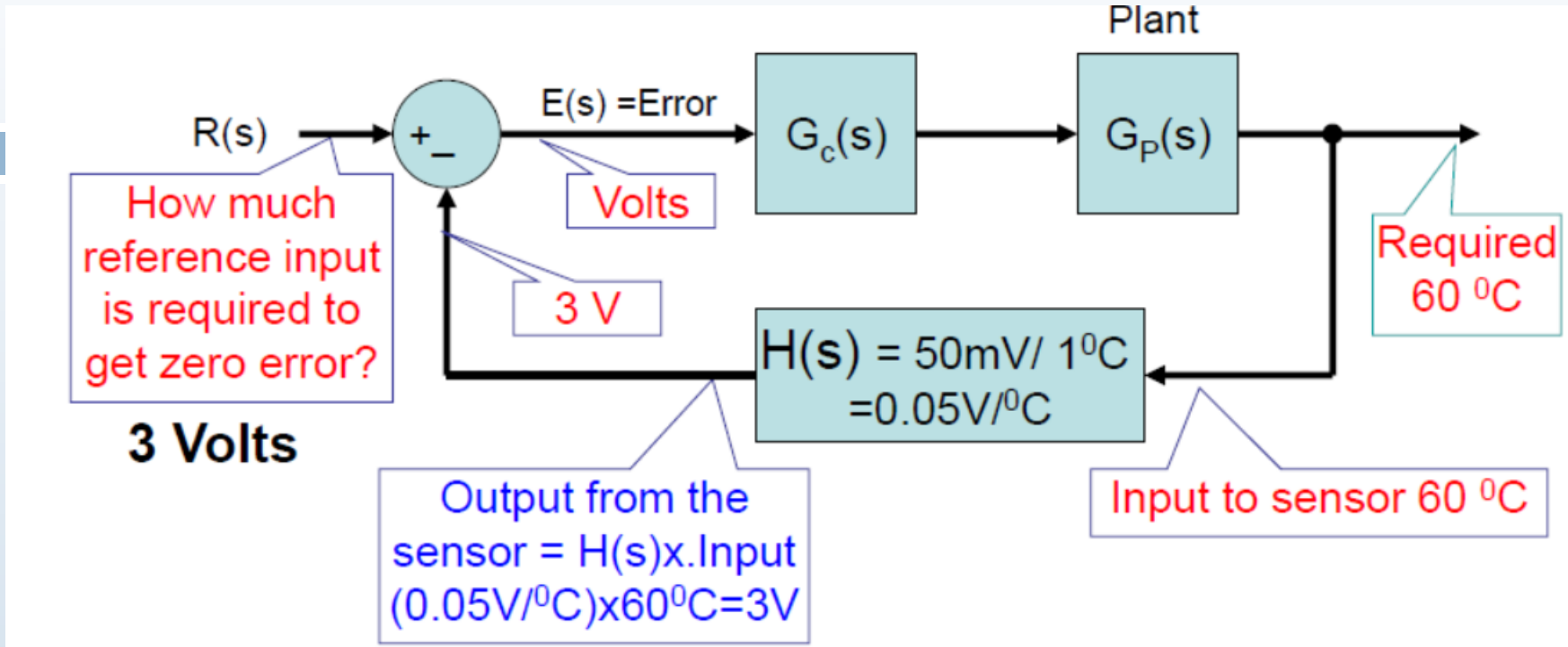
# Example

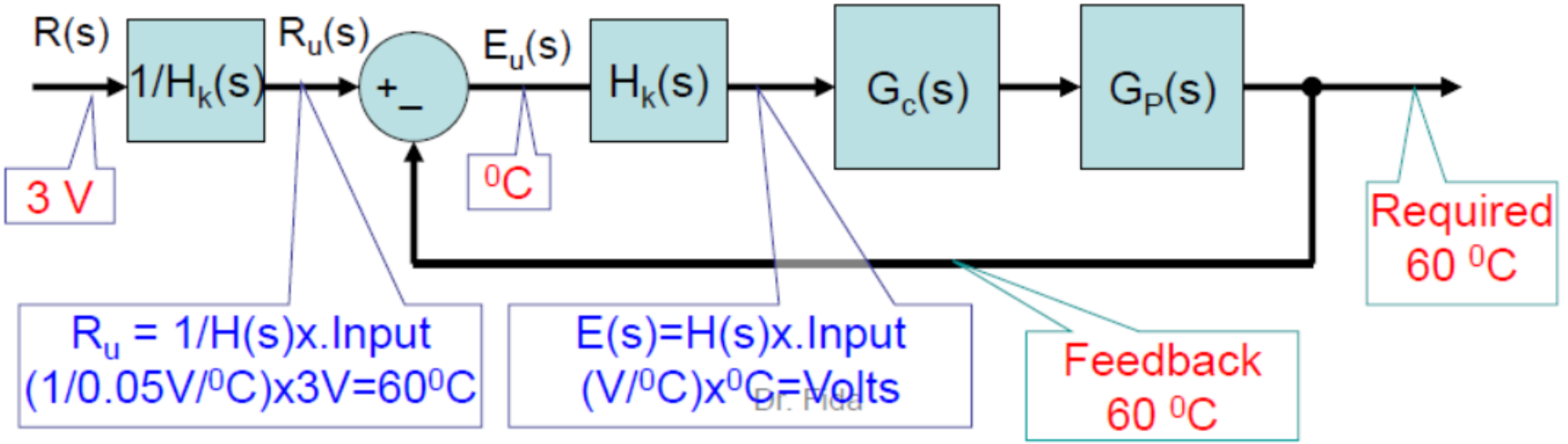





Transform to **UNITY FEEDBACK**




Plant
R(s)
+_
E(s) =Error
Volts
$G_c(s)$
$G_P(s)$
How much reference input is required to get zero error?
3 Volts
3 V
H(s) = 50mV/ 1$^0$C
=0.05V/$^0$C
Required 60 $^0$C
Output from the sensor = H(s)x.Input
(0.05V/$^0$C)x60$^0$C=3V
Input to sensor 60 $^0$C


R(s)
$1/H_k(s)$
$R_u(s)$
+_
$E_u(s)$
$H_k(s)$
$G_c(s)$
$G_P(s)$
3 V
$^0$C
Required 60 $^0$C
$R_u$ = 1/H(s)x.Input
(1/0.05V/$^0$C)x3V=60$^0$C
E(s)=H(s)x.Input
(V/$^0$C)x$^0$C=Volts
Feedback 60 $^0$C

# Quiz # 3 Solution



```
!pip install control numpy matplotlib

import control
import control.matlab as mpcontrolPC

# System
Ts = 0.05
num = [2]
den = [1, 1]
g = mpcontrolPC.tf(num, den)
N = 400

print(g)
gc = mpcontrolPC.tf(num, den)

print(gc)

# Step input signal
# The control.matlab library doesn't have a built-in step_signal function like this.
# You can create a step input using numpy:
import numpy as np
t = np.arange(0, N * Ts, Ts) # Create a time vector
x = np.ones_like(t) # Create a step signal of ones

# Calculate the output signal
y, t, x = mpcontrolPC.lsim(g, x, t)

# You can now plot the input and output signals if needed
import matplotlib.pyplot as plt
plt.plot(t, x, label='Input Signal')
plt.plot(t, y, label='Output Signal')
plt.xlabel('Time (s)')
```

```
dc_gain=control.dcgain(g)
print(dc_gain)
```

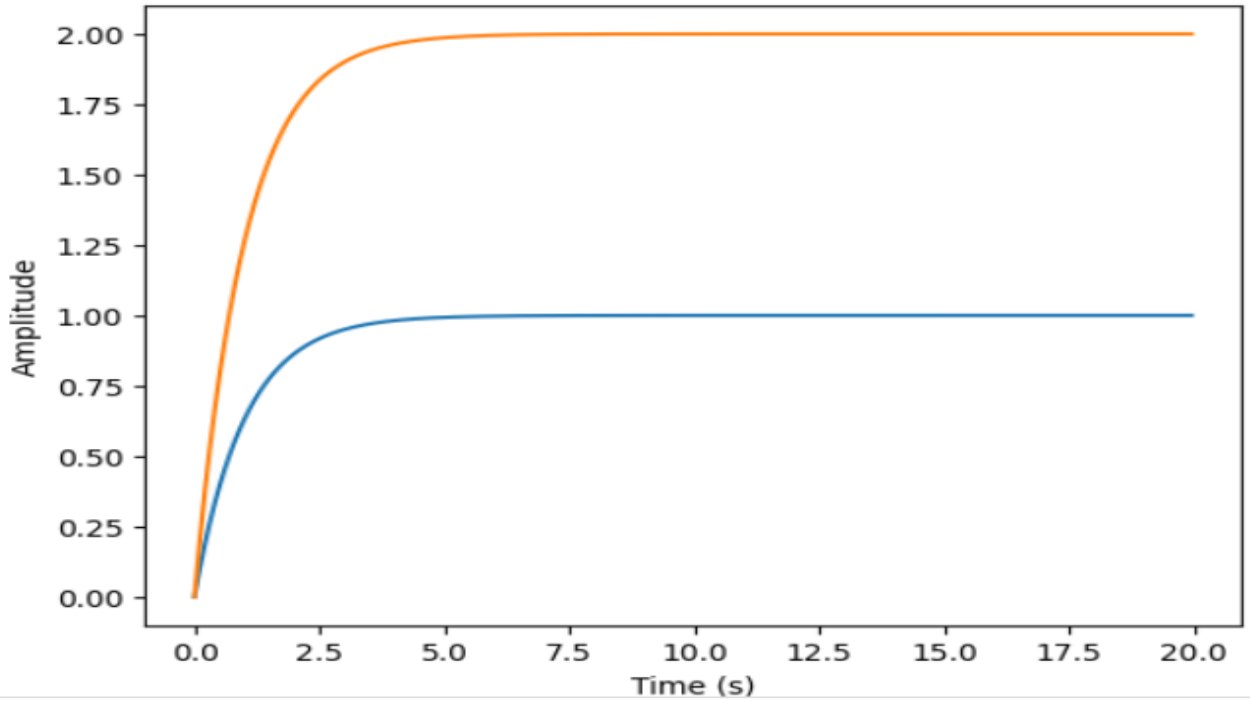
# DC gain is 2



```
Inputs (1): ['u[0]']
Outputs (1): ['y[0]']

    2
  -----
  s + 1
<TransferFunction>: sys[46]
Inputs (1): ['u[0]']
Outputs (1): ['y[0]']

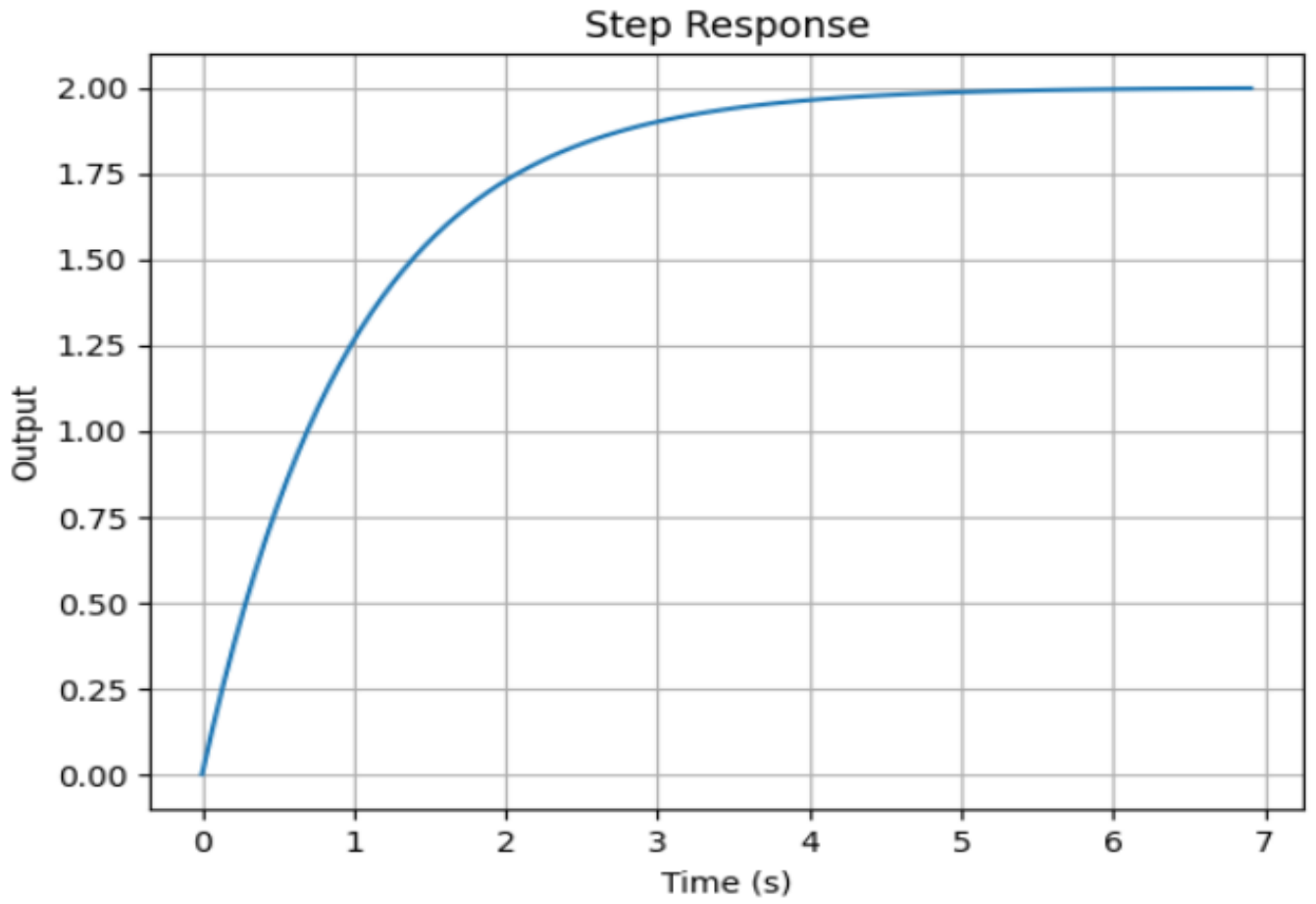
    2
  -----
  s + 1
2.0
```



# Time Constant(s)



```
# For a first-order system G(s) = K / (tau*s + 1), the time constant is tau.
# If your system is first-order, you can identify tau from the denominator coefficients.
# For higher-order systems, the concept of a single time constant is not well-defined,
# but you can analyze the poles of the system to understand the response speed.
# The time constant is related to the real part of the dominant pole(s).
```



# Python Demo

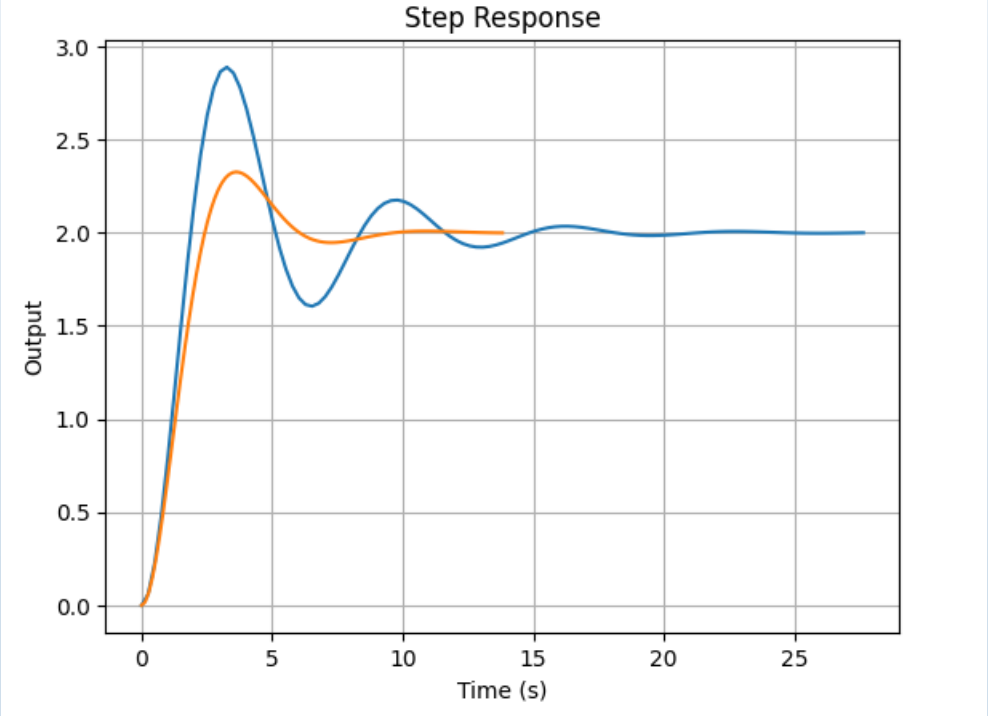
```
import control
N = 400
Ts=0.0005
t = [n*Ts for n in range(N)]
num = [2]
den = [1, 0.2, 5]
g = control.TransferFunction(num, den)
control.pole_zero_plot(g)
response = control.step_response(g, T=t)
response.plot()
dc_gain=control.dcgain(g)
Print(dc_gain)
```

# Effect of freq & damping



```
num = [2]
den = [1, 0.5, 1] # Represents 1/(s+1)
sys = control.tf(num, den)
den1 = [1, 1, 1] # Represents 1/(s+1),
sys1 = control.tf(num, den1)
# Calculate the step response
t, y = control.step_response(sys)
t1, y1 = control.step_response(sys1)

# Plot the step response
plt.plot(t, y)
plt.plot(t1, y1)
plt.xlabel('Time (s)')
plt.ylabel('Output')
plt.title('Step Response')
plt.grid(True)
plt.show()
```



- Laptop for Python Demo on Monday's class
  - 15/09/2025